

IOT basic, Number system and its conversions, data read from network.

1. What is network

Network means a group of devices interconnect with each other by a router.

In networking we have two types of IP addresses

*** ipv4 – 64 bit (192.168.0.127 -> 0 or 1 is bit so -> 11000000.
10101000.0. 1111111)**
*** ipv6 – 128 bit**

2. What are the type of data we have in network.

*** 1 Bit = small unit 0 or 1**
*** 1 byte = 8bit so example (01000001 -> 1byte -> ascii = A)**

3. Date examples

Binary: 11111111

Decimal: 255

Hex: FF

4. Three types of number system.

System	Base	digits
Binary	2	0,1

Decimal	10	0-9
Hexadecimal	16	0-9,A-F

5. Sample IOT data packet

0x7E 0x01 0x0A 0xFF

What Happens When You Send Data?

Application → OS → Network Card → Cable/WiFi → Router → Internet → Server

ASCII & Encoding

- Computers send **numbers**, not characters
- A → 65 (decimal) → 0x41 (hex)
- In java (byte) 0x41 we can convert hexa to decimal
- (char) decimal to character
- Integer.parseInt("10101",2) // binary to decimal conversion

IoT devices may send:

Temperature = 30

as:

0x33 0x30 // ASCII '3'0'

TCP vs UDP (EXTREMELY IMPORTANT)

♦ TCP (Reliable)

- Connection-oriented
- Guarantees delivery
- Slower

Used for:

- HTTP
- Database
- Banking
- Critical IoT data

♦ UDP (Fast)

- No guarantee
- Faster
- No connection

Used for:

- Video streaming
- GPS
- Sensor data (sometimes)

OSI Model (INTERVIEW FAVORITE)

Layer	Name	Example
7	Application	HTTP, MQTT
6	Presentation	Encoding, encryption
5	Session	Connection control
4	Transport	TCP / UDP
3	Network	IP
2	Data Link	MAC

1	Physical	Cable, WiFi
---	----------	-------------

TCP/IP vs OSI (Quick Mapping)

OSI Model	TCP/IP Model
Application	Application
Presentation	Application
Session	Application
Transport	Transport
Network	Internet
Data Link	Network Access
Physical	Network Access

TCP/IP Model Layers (Top → Bottom)

1 Application Layer

What it does:

- Provides network services to applications
- Combines OSI's Application, Presentation, and Session layers

Protocols:

- HTTP / HTTPS
- FTP
- SMTP, POP3, IMAP
- DNS

- MQTT (important for **IoT**)
- CoAP

Example:

Browser accessing a website, IoT device sending sensor data

Transport Layer

What it does:

- End-to-end communication
- Port numbers
- Reliability and flow control

Protocols:

- **TCP** – reliable, connection-oriented
- **UDP** – fast, connectionless

Example:

- TCP → HTTP, HTTPS
- UDP → DNS, video streaming, IoT telemetry

Internet Layer

What it does:

- Logical addressing
- Routing packets across networks

Protocols:

- IP (IPv4 / IPv6)

- ICMP (errors, ping)
- ARP (IP → MAC mapping)
- IPsec

Example:

Finding the best path from source to destination

Network Access Layer

(also called **Link Layer**)

What it does:

- Physical transmission of data
- MAC addressing
- Framing and error detection

Technologies / Protocols:

- Ethernet
- Wi-Fi
- Bluetooth
- LTE
- Physical cables (UTP, Fiber)

TCP 3-Way Handshake (Connection setup)

This happens **only when a TCP connection is created**:

1. **Client → SYN**
"I want to connect. My initial sequence number is X."
2. **Server → SYN-ACK**
"Okay. My sequence number is Y, and I acknowledge X."
3. **Client → ACK**
"I acknowledge Y."

Connection is now established

After this:

- Both sides know each other's sequence numbers
- TCP state becomes **ESTABLISHED**

Client → DATA (seq=101)

Server → ACK (ack=201)

Client → DATA (seq=201)

Server → ACK (ack=301)

PACKETS & FRAMES (IOT CORE)

Packet structure

| Header | Body | Footer |

Header:

- Device ID
- Command
- Length

Body:

- Sensor data

Footer:

- CRC / checksum

Data conversions

- 32 decimal to binary conversion

$$32 \% 2 \rightarrow 0, 32/2=16$$

$$16 \% 2 \rightarrow 0, 16/2=8$$

$$8 \% 2 \rightarrow 0, 8/2 = 4$$

$$4 \% 2 \rightarrow 0, 4/2 = 2$$

$$2 \% 2 \rightarrow 0, 2/2 = 1$$

$$1 \% 2 \rightarrow 1,$$

Binary to decimal

$$1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$= 32$$

Ex: 2

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 8 + 0 + 2 + 1$$

$$= 11$$

Binary → Hexadecimal

Rule

Group binary digits into **4 bits (nibbles)** from the right.

Binary → Hex table

0000 = 0 0100 = 4 1000 = 8 1100 = C
0001 = 1 0101 = 5 1001 = 9 1101 = D
0010 = 2 0110 = 6 1010 = A 1110 = E
0011 = 3 0111 = 7 1011 = B 1111 = F

0010 0000

0010 = 2

0000 = 0

Hexadecimal → Binary

Rule

Convert **each hex digit to 4-bit binary**

Example: $2F_{16} \rightarrow ?_2$

2 = 0010

F = 1111

$2F_{16} = 00101111_2$

Conversion	Method
Decimal → Binary	Divide by 2, read remainders bottom-up

Binary → Decimal	Multiply bits by powers of 2
Binary → Hex	Group into 4 bits
Hex → Binary	Each hex digit → 4 bits

Byte ↔ Decimal (VERY IMPORTANT IN JAVA)

Key rule

- Java byte is **signed**: -128 to 127
- Decimal values 128–255 need **unsigned conversion**

Byte → Decimal (unsigned)

```
byte b = (byte) 0xFF; // -1 in signed byte
int decimal = b & 0xFF;
```

```
System.out.println(decimal); // 255
```

✓ Used in file reading, network packets

Decimal → Byte

```
int decimal = 255;
byte b = (byte) decimal;
```

```
System.out.println(b); // -1
```

✓ Casting truncates to 8 bits

Byte **Hexadecimal**

Byte → Hex String (MOST COMMON)

Method 1 (Standard)

```
byte b = (byte) 255;
```

```
String hex = String.format("%02X", b & 0xFF);  
System.out.println(hex); // FF
```

Method 2 (Fast, common in libraries)

```
byte b = 10;  
String hex = Integer.toHexString(b & 0xFF).toUpperCase();
```

```
if (hex.length() == 1) hex = "0" + hex;  
System.out.println(hex); // 0A
```

Byte Array → Hex String (REAL WORLD)

```
byte[] data = {10, 15, -1};

StringBuilder sb = new StringBuilder();
for (byte b : data) {
    sb.append(String.format("%02X", b & 0xFF));
}

System.out.println(sb.toString()); // 0A0FFF
```

Hex String → Byte

```
String hex = "FF";
byte b = (byte) Integer.parseInt(hex, 16);

System.out.println(b); // -1
```

Hex String → Byte Array

```
String hex = "0A0FFF";
byte[] bytes = new byte[hex.length() / 2];

for (int i = 0; i < bytes.length; i++) {
    bytes[i] = (byte) Integer.parseInt(
        hex.substring(i * 2, i * 2 + 2), 16
    );
}
```

Decimal **Hexadecimal**

Decimal → Hex

```
int dec = 255;  
String hex = Integer.toHexString(dec).toUpperCase();  
  
System.out.println(hex); // FF
```

Hex → Decimal

```
String hex = "FF";  
int dec = Integer.parseInt(hex, 16);  
  
System.out.println(dec); // 255
```

Byte **Binary**

Byte → Binary String

```
byte b = 5;  
  
String bin = String.format("%8s",  
    Integer.toBinaryString(b & 0xFF))  
    .replace(' ', '0');
```

```
System.out.println(bin); // 00000101
```

Binary → Byte

```
String bin = "11111111";  
byte b = (byte) Integer.parseInt(bin, 2);
```

```
System.out.println(b); // -1
```

Byte Readable Format (String / Text)

String → Byte Array

```
String text = "Hello";  
byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
```

Byte Array → String

```
String text = new String(bytes, StandardCharsets.UTF_8);
```

✓ Used in:

- File I/O
- HTTP

- JSON
- Databases

6 ALL IMPORTANT JAVA PARSING METHODS (CHEAT SHEET)

Conversion	Java Method
String → int	Integer.parseInt(str)
String → int (base N)	Integer.parseInt(str, radix)
int → String	Integer.toString(n)
int → Hex	Integer.toHexString(n)
int → Binary	Integer.toBinaryString(n)
byte → unsigned int	b & 0xFF
String → byte[]	getBytes(UTF_8)
byte[] → String	new String(bytes, UTF_8)

7 How BIG JAVA Applications Do This

Example: Network packet decoding

```
byte[] packet = receivePacket();
```

```
int length = packet[0] & 0xFF;
```

```
String payloadHex = bytesToHex(packet);
```

Example: File parsing

```
byte[] fileData = Files.readAllBytes(path);  
int header = fileData[0] & 0xFF;
```